Universidad de La Habana
Facultad de Matemática y Computación

Ingeniería de Software | 3.er año | Curso 2026–2027

ACTIVIDAD

Seminario de Arquitecturas de Software:

Un Análisis Comparativo de Estilos Basados en Capas y Dominio

EQUIPO

Team Bales

INTEGRANTES Y PREGUNTAS EN LAS QUE TRABAJÓ

Heily Rodríguez Rodríguez ▪ 1 y 2

Enrique González Moreira ▪ 4 y 6

Ernesto Alejandro Soler Choong ▪ 3 y 5

Mauricio Brindon Carbo ▪ Sin contribución

REPOSITORIO PÚBLICO

https://github.com/Team-Bales/seminario-arq-capas

Índice

Introducción **2**

1. P1: Trade-offs N-Capas vs. Clean/Onion/Hexagonal **2**
1.1. El error fundamental de acoplamiento 2
1.2. Solución estructural: Regla de Dependencia y Puertos y Adaptadores 2
1.3. Impacto en la testabilidad y en el cambio de SQL Server a MongoDB 3

2. P2: Simetría del Hexágono vs. Jerarquía de Capas **4**
2.1. Problema conceptual que resuelve la simetría del hexágono 4
2.2. Traducción de «Puerto y Adaptador» a las interfaces internas de Onion . . . 4
2.3. Por qué el hexágono no tiene seis lados por una razón numérica 6

3. P3: DDD y Bounded Context: Sistema de Seguros de Automóviles **6**
3.1. División del sistema en contextos delimitados 6
3.2. Protección del modelo de «Póliza» 8
3.3. Comunicación desacoplada entre contextos 9
3.4. Relación con Onion/Clean y el Adaptador de Servicio 12

4. P4: Evaluación de la Inversión de Dependencias en la Práctica: OrderProcessor **16**
4.1. a) ¿Por qué viola la dependencia directa los principios de Hexagonal, Limpia y Onion? 16
4.2. b) Rediseño con inversión de dependencias 17
4.3. c) Pruebas aisladas sin HTTP y mantenibilidad al cambiar de proveedor . . 19

5. P5: Análisis del Kitty Problem y el Principio Abierto/Cerrado **20**
5.1. Por qué la división por verbos o entidades provocó el *Kitty Problem* 20
5.2. Diseño con DDD y Onion/Clean: un solo componente afectado 22
5.3. Solución con `Kittens` y `Rides` como *plugins* 24

6. P6: Evaluación de N-Capas frente a Atributos de Calidad: Open Layers y Sinkhole Anti-pattern **29**
6.1. Capas abiertas, aislamiento de capas y mantenibilidad a largo plazo 29
6.2. Capas abiertas, sinkhole anti-pattern y la regla del 80/20 30
6.3. Hexagonal y Limpia frente a N-Capas para alta escalabilidad y agilidad . . . 31

Conclusiones **32**

Referencias bibliográficas **33**

Introducción

Este documento recoge las respuestas del equipo Team Bales a las seis preguntas del seminario. Cada capítulo responde a una pregunta a partir de la bibliografía orientada y termina con las notas de investigación que la sustentan. Las preguntas se abordan de forma independiente, pero comparten un mismo enfoque: analizar cómo se organizan las dependencias entre las partes de un sistema y qué consecuencias tiene esa organización para su mantenimiento y sus pruebas. Al final se incluyen la bibliografía y la declaración de uso de inteligencia artificial.

P1: Trade-offs N-Capas vs. Clean/Onion/Hexagonal

1.1. El error fundamental de acoplamiento

El error está en la **dirección de las dependencias**. En una arquitectura N-Capas tradicional, las dependencias del código fuente siguen el flujo de control: la presentación llama al negocio y el negocio llama a los datos. Como para llamar a un módulo hay que nombrarlo en el código, el negocio termina dependiendo de la base de datos (Martin, 2017, pp. 44–45). Esto invierte lo que debería ocurrir. La lógica de negocio es la parte más valiosa y estable del sistema, pero queda atada a la persistencia, que es la que más cambia. Martin advierte que un componente que se espera volátil no debe ser objeto de dependencia de otro difícil de cambiar (Martin, 2017, p. 120), y que un componente estable debe ser también abstracto (Martin, 2017, p. 126). Cockburn identifica el mismo defecto y lo llama *enredo* entre la lógica de negocio y la interacción con el exterior (Cockburn, 2005, «Nature of the Solution»).

El problema se agrava porque con las dependencias también viajan los datos. Los formatos propios de la base de datos, como las filas devueltas por una consulta, atraviesan las capas, cuando Martin advierte que estas estructuras no deben cruzar las fronteras hacia dentro (Martin, 2017, pp. 204, 207). De este modo, el esquema de la base de datos se convierte en el modelo de todo el sistema. Por eso un cambio en una tabla obliga a modificar el negocio y, a través de él, la interfaz de usuario, que es justo lo que le sucede al equipo del enunciado.

1.2. Solución estructural: Regla de Dependencia y Puertos y Adaptadores

La **Regla de Dependencia** de Martin establece que las dependencias del código fuente solo pueden apuntar hacia dentro, hacia las políticas de más alto nivel. Nada de lo declarado en un círculo externo puede ser nombrado desde uno interno (Martin, 2017, p. 203). Para

lograrlo se aplica la inversión de dependencias: el núcleo declara una interfaz, por ejemplo `IInventarioRepository`, y la capa de infraestructura la implementa. En ejecución, el negocio sigue invocando a la persistencia, pero en el código es la persistencia la que depende del negocio (Martin, 2017, pp. 45, 206). La interfaz de acceso a datos queda en el componente de negocio, mientras que su implementación queda en el de base de datos, que pasa a ser un *plugin* de las reglas de negocio (Martin, 2017, pp. 46, 168).

Los **Puertos y Adaptadores** de Cockburn llegan a la misma estructura con otro vocabulario. Sustituyen la asimetría arriba/abajo de las capas por una asimetría dentro/fuera. Un *puerto* es una conversación con un propósito concreto, expresada como una API. Un *adaptador* traduce esa API a una tecnología determinada, y un mismo puerto puede tener varios adaptadores: uno para SQL, otro para ficheros o una base de datos simulada en memoria (Cockburn, 2005, «Nature of the Solution»). La base de datos es un actor secundario, es decir, uno al que la aplicación conduce, y se conecta a través de un puerto secundario (Cockburn, 2005, «The Left-Right Asymmetry»). En los dos enfoques ocurre lo mismo: la interfaz de persistencia pertenece al núcleo y la infraestructura depende de él.

1.3. Impacto en la testabilidad y en el cambio de SQL Server a MongoDB

Respecto a la testabilidad, con esta estructura las reglas de negocio pueden probarse sin interfaz de usuario, sin base de datos y sin ningún otro elemento externo (Martin, 2017, p. 202), que es también el objetivo declarado del patrón hexagonal (Cockburn, 2005, «Intent»). Basta con inyectar un doble de prueba, como un *stub*, un *mock* o un repositorio en memoria, en lugar del adaptador real. La guía de Microsoft describe esta práctica con repositorios falsos (De la Torre Llorente et al., 2010, p. 146), y Martin señala que los casos de uso son testables porque sus pasarelas a la base de datos pueden sustituirse por *stubs* (Martin, 2017, p. 214). El resultado es que las pruebas dejan de ser lentas, porque evitan recursos costosos como la base de datos (Martin, 2017, p. 252), y dejan de ser frágiles, porque siguen la regla de no depender de cosas volátiles (Martin, 2017, p. 251).

En cuanto al cambio de SQL Server a MongoDB, basta con escribir un nuevo adaptador que implemente el mismo puerto y registrarlo en lugar del anterior. El dominio y los casos de uso no se modifican. Martin menciona este caso entre las propiedades comunes a estas arquitecturas, entre ellas la hexagonal: se puede cambiar SQL Server por Mongo u otra base de datos sin que las reglas de negocio queden atadas a ella (Martin, 2017, p. 202). La guía de Microsoft coincide: si las interfaces de los repositorios se definen en el dominio, la persistencia puede sustituirse sin afectarlo (De la Torre Llorente et al., 2010, pp. 145–146). Para que esto funcione, las entidades deben ser ignorantes de la persistencia y por la frontera solo deben cruzar estructuras simples (Martin, 2017, p. 207; De la Torre Llorente et al., 2010, p. 55). Conviene añadir un matiz honesto: Martin advierte que estos reemplazos pueden no ser triviales, aunque la estructura de *plugins* los hace factibles (Martin, 2017, p. 171).

P2: Simetría del Hexágono vs. Jerarquía de Capas

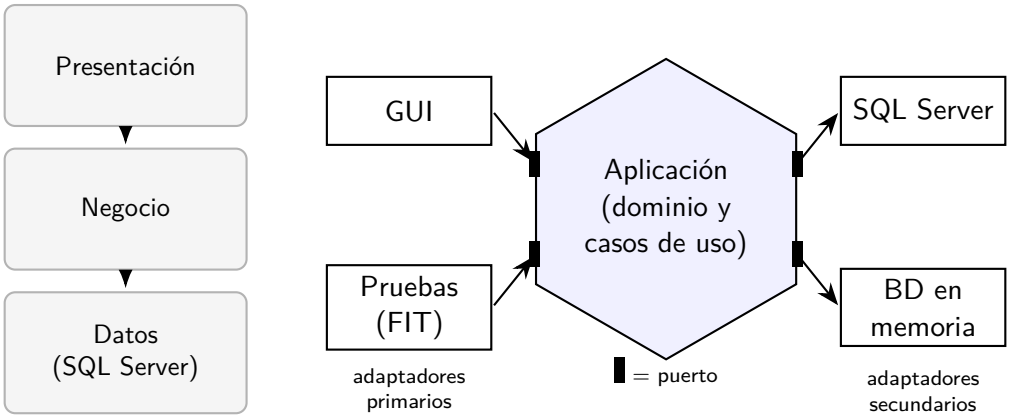
2.1. Problema conceptual que resuelve la simetría del hexágono

Hexagonal y Onion persiguen lo mismo, aislar el núcleo de negocio, y el propio Palermo reconoce que ambas comparten la premisa de externalizar la infraestructura y escribir código adaptador para no acoplarse a ella (Palermo, 2008a). Lo que cambia es el dibujo, y el dibujo importa porque transmite una idea de cómo se organiza el sistema. Cuando una arquitectura se dibuja como una pila de capas, el dibujo sugiere un **orden de importancia**: la interfaz de usuario queda arriba y la base de datos abajo, como si fuera el cimiento sobre el que se apoya todo. Con esa imagen, que la lógica de negocio dependa de la base de datos parece lo normal. Cockburn explica que este dibujo aparece porque muchas aplicaciones solo se comunican con dos cosas: el usuario y la base de datos. Pero tiene dos problemas. Primero, en la práctica nadie respeta las líneas que separan las capas, y la lógica de negocio acaba repartida entre ellas. Segundo, si la aplicación se comunica con más de dos cosas, ya no cabe en una sola columna (Cockburn, 2005, «Nature of the Solution»). Palermo llega a lo mismo por otro camino: si se redibuja la N-Capas tradicional como círculos, se ve que toda la aplicación gira alrededor del acceso a datos (Palermo, 2008c).

El hexágono cambia la pregunta: ya no importa qué está arriba y qué está abajo, sino **qué está dentro de la aplicación y qué está fuera** (Figura 2.1). Cockburn observa que los problemas con la interfaz de usuario y con la base de datos tienen la misma causa: la lógica de negocio está mezclada con el código que se comunica con el exterior (Cockburn, 2005, «Motivation», «Nature of the Solution»). Por eso el hexágono los trata igual. La interfaz de usuario, las pruebas y la base de datos son, todos, elementos externos que se conectan a la aplicación. Onion se acerca a esta idea, porque pone la interfaz de usuario, la infraestructura y las pruebas en el mismo anillo exterior y deja claro que la base de datos «no es el centro» (Palermo, 2008a). La diferencia es que Onion sigue ordenando por niveles lo que hay *dentro* del núcleo, mientras que el hexágono se fija sobre todo en la frontera entre dentro y fuera.

2.2. Traducción de «Puerto y Adaptador» a las interfaces internas de Onion

En la Arquitectura Hexagonal, un **puerto** es la «puerta» por la que la aplicación conversa con el exterior para un fin concreto, y se define como una interfaz o API. Un **adaptador** es la pieza que conecta una tecnología concreta, como SQL Server o una pantalla, a ese puerto (Cockburn, 2005, «Nature of the Solution»). Onion convierte esta idea en una regla sencilla: las capas de dentro definen las interfaces y las capas de fuera las implementan, de modo que



(a) Pila de capas **(b) Hexágono**

Figura 2.1: De la pila de capas al hexágono. En (a) la posición marca una jerarquía y la base de datos es el cimiento; en (b) todos los actores externos están al mismo nivel y se conectan por puertos. Elaboración propia a partir de Cockburn (2005, «Nature of the Solution»).

las dependencias siempre apuntan hacia el centro (Palermo, 2008c). El ejemplo más claro es el **repositorio**, la pieza que guarda y recupera los datos. Su interfaz se coloca en el anillo que rodea al modelo de dominio, y la clase que de verdad accede a la base de datos se coloca en el borde. Además, la interfaz se diseña según lo que necesita el núcleo, no según la tecnología que habrá detrás (Palermo, 2008a). Dicho con el vocabulario de Cockburn, la interfaz del repositorio es un **puerto secundario** y su implementación, un **adaptador secundario** (Figura 2.2).

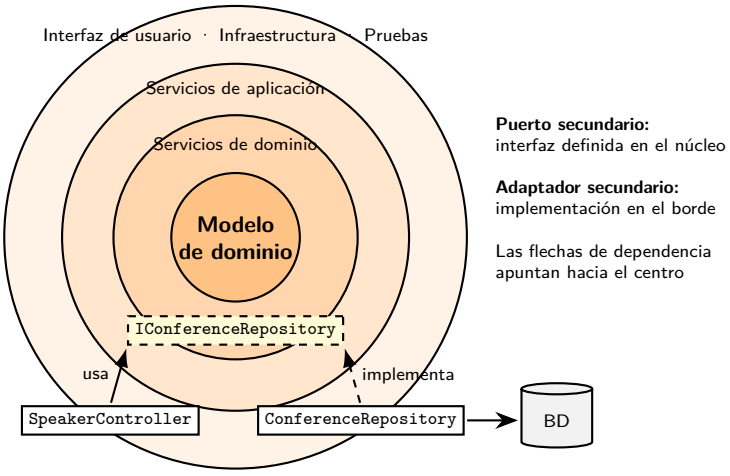


Figura 2.2: Puerto y adaptador en Onion, con el ejemplo de CodeCampServer. La interfaz de repositorio vive en el núcleo y su implementación en el anillo exterior; un contenedor IoC la inyecta en el controlador. Elaboración propia a partir de Palermo (2008a) y Palermo (2008b).

Palermo lo muestra con su aplicación CodeCampServer. El controlador de la interfaz, `SpeakerController`, no usa directamente la clase que accede a la base de datos, sino la interfaz `IConferenceRepository`, que está en el núcleo. La clase `ConferenceRepository` implementa

esa interfaz desde fuera, y un contenedor de inversión de control (IoC) se encarga de crearla y entregársela al controlador en tiempo de ejecución (Palermo, 2008b). Cockburn hace lo mismo en su ejemplo: la clase `Discounter` recibe en su constructor un `RateRepository`, que puede ser la base de datos real o una simulada en memoria (Cockburn, 2005, «Sample Code»). Los dos autores apoyan este mecanismo en el principio de inversión de dependencias (Palermo, 2008a; Cockburn, 2005, «Related Patterns»). La diferencia es que Onion se centra sobre todo en el lado de la infraestructura, ya que cualquier capa externa puede llamar directamente a una interna (Palermo, 2008c). El hexágono, en cambio, trata igual los dos lados: lo que pone en marcha la aplicación y lo que la aplicación utiliza.

2.3. Por qué el hexágono no tiene seis lados por una razón numérica

Cockburn aclara que el número seis no tiene ningún significado. Eligió un hexágono para tener espacio donde dibujar todos los puertos y adaptadores que hagan falta, algo que un dibujo de capas en una sola columna no permite. Con esa forma quiere resaltar dos cosas: la diferencia entre dentro y fuera, con todos los puertos de la misma naturaleza, y que la aplicación tiene un número concreto de puertos distintos (Cockburn, 2005, «Nature of the Solution»). De hecho, considera que el número de puertos, normalmente entre dos y cuatro, es «en gran medida una cuestión de gusto» (Cockburn, 2005, «How Many Ports?»). Que el hexágono tenga seis lados no significa que la aplicación tenga seis puertos.

Lo que sí importa es que en un hexágono **ningún lado está por encima de otro**. Todos los lados son iguales, así que la posición deja de indicar importancia, que es justo lo que ocurría en la pila de capas. Por eso Cockburn propone olvidar «por un momento» las diferencias entre izquierda y derecha o entre arriba y abajo (Cockburn, 2005, «Nature of the Solution»). Más adelante sí distingue dos lados. A la izquierda pone lo que pone en marcha la aplicación, los **puertos primarios**, como la interfaz de usuario o las pruebas. A la derecha pone lo que la aplicación utiliza, los **puertos secundarios**, como la base de datos. Pero esa distinción depende de quién empieza la conversación, no de cuál es más importante (Cockburn, 2005, «The Left-Right Asymmetry»). En resumen, el hexágono elimina la jerarquía y solo conserva la diferencia que importa: lo que está dentro de la aplicación y lo que está fuera.

P3: DDD y Bounded Context: Sistema de Seguros de Automóviles

3.1. División del sistema en contextos delimitados

La solución ingenua sería construir un único modelo de «Póliza» compartido por todas las áreas. Ese modelo acumularía cláusulas, planes de cobro y datos de peritaje, y cualquier cambio pedido por un área afectaría a todas las demás. Domain-Driven Design (DDD) propone lo contrario: expresar el modelo en el lenguaje de los expertos del negocio, el llamado *Lenguaje*

Ubicuo, y está indicado precisamente para aplicaciones empresariales complejas (De la Torre Llorente et al., 2010, p. 21). Desde esta perspectiva, que «póliza» signifique algo distinto para el actuario, para facturación y para el ajustador no es un defecto que haya que corregir, sino la señal de que existen varios modelos legítimos.

El contexto delimitado es un concepto propuesto por Evans (como se citó en Richards y Ford, 2020, cap. 7): todo lo relativo a un dominio es visible dentro del contexto y opaco fuera de él. En vez de crear una única clase compartida por toda la organización, cada dominio define la suya y las diferencias se concilian en los puntos de integración (Richards y Ford, 2020, cap. 7). Para trazar las fronteras en el caso de la aseguradora se aplican cuatro criterios:

C1. Lenguaje. Cada área cuyos expertos usan un vocabulario propio es candidata a un contexto propio. El criterio se deriva de que el modelo debe expresarse en el Lenguaje Ubicuo de los expertos (De la Torre Llorente et al., 2010, p. 21) y de que cada entidad funciona mejor dentro de un contexto localizado (Richards y Ford, 2020, cap. 7).

C2. Partición por dominio, no técnica. Si los componentes se organizan por capas técnicas, cada flujo de negocio queda repartido entre todas ellas. Si se organizan por dominio, la estructura refleja mejor los cambios que realmente ocurren (Richards y Ford, 2020, cap. 8).

C3. Granularidad. Cada contexto debe tener un propósito funcionalmente cohesivo. Las entidades que necesitan cooperar en una misma transacción sugieren una frontera común. Si dos contextos necesitan comunicarse en exceso, puede considerarse fusionarlos (Richards y Ford, 2020, cap. 17).

C4. Características operativas. Un *cuanto arquitectónico* es un artefacto desplegable de forma independiente, con alta cohesión funcional y connascencia síncrona. Una llamada síncrona obliga a que las dos partes tengan las mismas características operativas mientras dura la llamada (Richards y Ford, 2020, cap. 7). Por eso conviene no acoplar de forma síncrona partes con exigencias operativas muy distintas.

Con estos criterios se proponen cuatro contextos delimitados, resumidos en la Tabla 3.1. Cada uno tiene su propio significado de «Póliza».

Decisión de diseño: Peritaje como contexto propio. La decisión menos evidente es separar Peritaje de Siniestros, aunque ambos participan en el mismo proceso. Hay tres argumentos a favor:

- Su lenguaje es distinto: inspección y valoración frente a expediente y reserva (criterio C1).
- No comparten transacción: el informe pericial es un producto terminado que se entrega a Siniestros (criterio C3).
- Sus características operativas difieren: Peritaje es una aplicación móvil usada en campo, posiblemente con conectividad intermitente, y acoplarla de forma síncrona a Siniestros le impondría las mismas exigencias (criterio C4).

Tabla 3.1: Contextos delimitados propuestos y significado de «Póliza» en cada uno.

Contexto	**Responsabilidad**	**Lenguaje propio**	**Qué es «Póliza»**
Suscripción y Pólizas (actuarios)	Diseñar, emitir, modificar, renovar, suspender y cancelar pólizas	cobertura, cláusula, riesgo, prima técnica, vigencia	Contrato con coberturas y cláusulas; raíz del agregado principal
Facturación de Primas	Emitir recibos, cobrar primas y gestionar impagos	recibo, plan de pagos, vencimiento, impago	Fuente de ingresos: cuenta de cobro asociada a un número de póliza
Siniestros	Abrir y tramitar expedientes y decidir indemnizaciones	expediente, declaración, reserva, indemnización	Cobertura aplicable en la fecha del siniestro
Peritaje (ajustadores)	Recibir solicitudes de peritaje, asignar ajustadores, evaluar daños en el lugar del accidente y emitir el informe pericial	inspección, daño, valoración, informe pericial	Número de póliza usado como mera referencia del vehículo asegurado

La alternativa de integrarlo en Siniestros sería razonable en una organización pequeña. Ningún estilo de partición es intrínsecamente más correcto que otro (Richards y Ford, 2020, cap. 8), y la granularidad adecuada se alcanza iterando (Richards y Ford, 2020, cap. 17).

Conviene distinguir esta división de los *Módulos* que propone la Guía de Arquitectura N-Capas: particiones verticales del modelo de dominio por áreas de negocio, con alta cohesión y comunicadas mediante interfaces (De la Torre Llorente et al., 2010, pp. 71–72). Un contexto delimitado va un paso más allá. No es un módulo de un modelo compartido: es un modelo propio, con sus propios datos.

3.2. Protección del modelo de «Póliza»

El aislamiento del modelo se consigue combinando cinco mecanismos.

M1. Un modelo de Póliza por contexto. No existe una clase «Póliza» compartida. Cada contexto incluye sus propias clases y esquemas, y nunca se acopla a la definición de clases ni a la base de datos de otro contexto (Richards y Ford, 2020, cap. 17). Solo Suscripción conoce cláusulas y coberturas. Siniestros maneja una *cobertura aplicable* y Facturación una *cuenta de cobro*.

M2. Aceptar la duplicación accidental. Martin (2017, pp. 154–155) distingue la duplicación verdadera, en la que todo cambio en una copia exige el mismo cambio en las demás, de la accidental, en la que las copias evolucionan por caminos distintos. Unificar la duplicación accidental crea un acoplamiento difícil de deshacer. Ambos contextos, Suscripción y Siniestros, manejan datos de cobertura (límites, franquicia, vigencia).

Suscripción los modela como cláusulas de un contrato que se negocian y renuevan; Siniestros solo necesita una instantánea para resolver un expediente. Las dos estructuras cambiarán por razones distintas, así que su parecido es accidental. Cuando el objetivo es el desacoplamiento, se prefiere la duplicación a la reutilización (Richards y Ford, 2020, cap. 17). El número de póliza, en cambio, no es una duplicación: es un identificador compartido que sirve de referencia entre contextos (mecanismo M3).

M3. Fronteras de agregado y referencias por identidad. Un agregado es un conjunto de objetos asociados que se trata como una unidad frente a los cambios de datos. Tiene una entidad raíz como único punto de entrada, y desde fuera solo puede referenciarse esa raíz, lo que asegura la integridad de los datos (De la Torre Llorente et al., 2010, p. 172). En Suscripción, la Póliza es la raíz de un agregado que contiene sus coberturas y cláusulas. Por analogía, la propuesta aplica la misma regla entre contextos: los demás contextos se refieren a la póliza solo por su número de póliza, nunca por su estructura interna.

M4. Datos aislados. Cada contexto tiene su propio almacenamiento y no se integra a través de esquemas compartidos (Richards y Ford, 2020, cap. 17).

M5. Traducción en la frontera. La información que llega de otro contexto entra solo a través de un componente que la convierte al modelo local. Los Agentes de Servicios externos de la Guía cumplen este papel: entre otras tareas, realizan mapeos básicos entre el formato de datos del servicio externo y el que usa la propia aplicación (De la Torre Llorente et al., 2010, pp. 108, 155). Un adaptador cumple la misma función en la Arquitectura Hexagonal: la aplicación interactúa con él sin conocer la naturaleza de lo que hay al otro lado (Cockburn, 2005). En la terminología original de DDD, este mecanismo de traducción defensiva se conoce como *Anti-Corruption Layer*; el término se menciona solo como orientación, porque no procede de las fuentes consultadas.

3.3. Comunicación desacoplada entre contextos

Richards y Ford (2020, cap. 14) distinguen dos modelos de interacción. El modelo basado en peticiones es el adecuado para consultas de datos estructuradas que necesitan certeza y control del flujo. El modelo basado en eventos es el adecuado para acciones que requieren capacidad de respuesta, escalabilidad y adaptabilidad, a cambio de admitir solo consistencia eventual. La propuesta combina ambos (Figura 3.1).

Eventos de dominio asíncronos como mecanismo por defecto. Cada contexto anuncia lo que ha hecho y los demás reaccionan, sin un coordinador central. Es la topología *broker*, en la que los procesadores publican eventos en canales de un intermediario ligero mediante publicación y suscripción (Richards y Ford, 2020, cap. 14). La coreografía, es decir, la ausencia de un mediador global, respeta la filosofía del contexto delimitado (Richards y Ford, 2020, cap. 17). La difusión de eventos es probablemente el mayor grado de desacoplamiento posible, porque el productor desconoce quién recibe el mensaje y qué hace con él (Richards y Ford, 2020, cap. 14). La Tabla 3.2 detalla los flujos principales. No pretende ser exhaustiva: otros

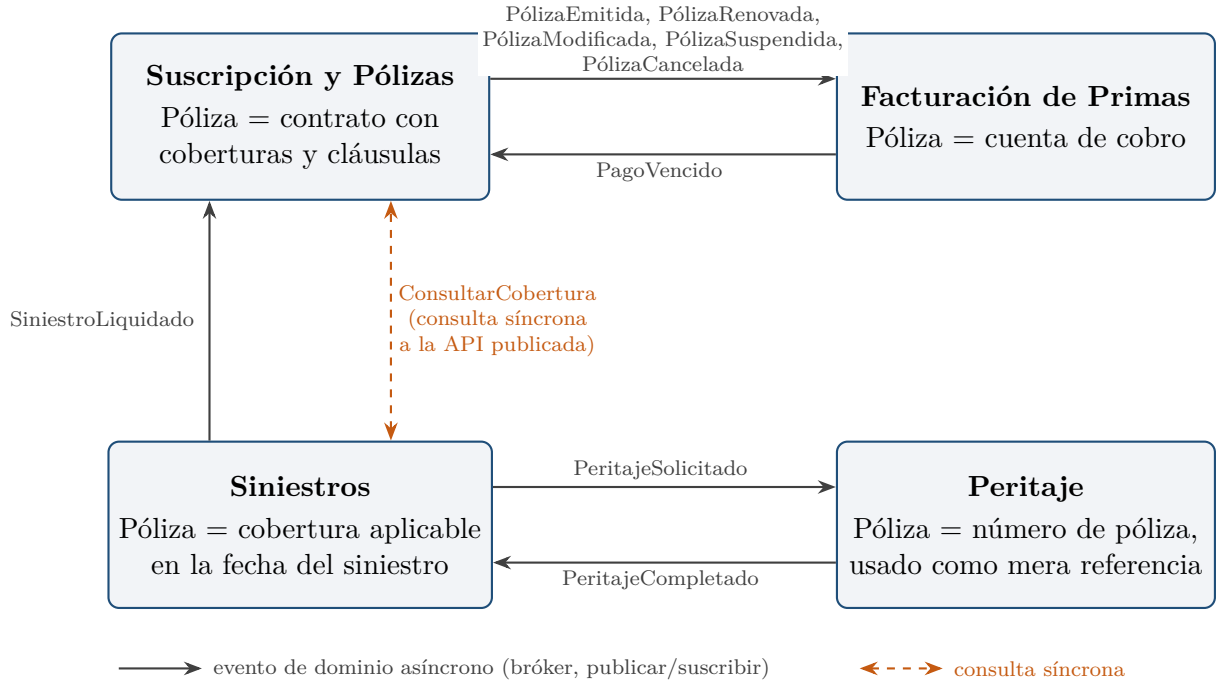


Figura 3.1: Mapa de contextos propuesto. Los eventos se publican en un bróker de mensajes; la flecha discontinua es la única interacción síncrona.

casos, como la regularización de un pago o la cancelación de un peritaje, seguirían el mismo patrón.

Dos decisiones de la tabla merecen justificación:

- **Cada contexto conserva la propiedad de sus decisiones.** Facturación no cambia el estado de una póliza: publica `PagoVencido` y es Suscripción quien decide la suspensión aplicando las cláusulas del contrato. Así, las reglas contractuales no se filtran a Facturación y los datos de cobro no se filtran a la Póliza.
- **Peritaje recibe lo que necesita.** El evento `PeritajeSolicitado` incluye la referencia de póliza y los datos del vehículo y del lugar que el asegurado aportó al declarar el siniestro. Así, el ajustador no depende de una consulta síncrona desde el lugar del accidente.

Extensibilidad. Que cada contexto anuncie lo que hizo, aunque hoy nadie lo escuche, facilita la extensibilidad (Richards y Ford, 2020, cap. 14). Por ejemplo, un futuro contexto de detección de fraude podría suscribirse a los eventos de Siniestros sin modificar ese contexto.

Consulta síncrona solo donde se necesita una respuesta inmediata. Al abrir un expediente, Siniestros necesita saber, con la información más actualizada posible, si existía cobertura en la fecha del accidente. Es una consulta de datos determinista, el caso para el que se recomienda el modelo basado en peticiones (Richards y Ford, 2020, cap. 14). En el diseño se implementa como una consulta síncrona a la API publicada de Suscripción (Listado 3.1).

Tabla 3.2: Interacciones entre contextos.

Mensaje	**Publica**	**Consume**	**Reacción del consumidor**
PólizaEmitida, PólizaRenovada, PólizaModificada, PólizaSuspendida, PólizaCancelada	Suscripción	Facturación	Crea, ajusta, suspende o cierra el plan de cobro; el evento incluye el número de póliza, la prima y el calendario de pagos
PagoVencido	Facturación	Suscripción	Decide, según las cláusulas del contrato, si suspende la póliza
PeritajeSolicitado	Siniestros	Peritaje	Asigna un ajustador; el evento incluye la referencia de póliza y los datos del vehículo y del lugar recogidos en la declaración del siniestro
PeritajeCompletado	Peritaje	Siniestros	Incorpora la valoración de daños al expediente
SiniestroLiquidado	Siniestros	Suscripción	Actualiza el historial de siniestralidad que se usa en las renovaciones
ConsultarCobertura (síncrona)	Siniestros pregunta	Suscripción responde	Verifica la cobertura vigente en la fecha del siniestro al abrir el expediente

Dentro de una arquitectura dirigida por eventos, la alternativa equivalente sería la mensajería petición–respuesta (Richards y Ford, 2020, cap. 14). Se prefiere la API porque, para una única interacción, evita añadir colas de petición y de respuesta. Otra alternativa sería mantener en Siniestros una réplica local de las coberturas, alimentada por los eventos de póliza. Eliminaría la dependencia temporal, pero la verificación también pasaría a ser eventualmente consistente; el diseño prioriza la información actualizada y deja la réplica como evolución posible. Esta es la única interacción síncrona del diseño; su coste se analiza en la Sección 3.4.

Sin transacciones distribuidas. Emitir una póliza y crear su plan de cobro no forman una transacción atómica. Construir transacciones entre servicios viola el principio de desacoplamiento, y la recomendación es revisar la granularidad antes que coordinarlas; las sagas son la excepción, con costes importantes (Richards y Ford, 2020, cap. 17). El sistema acepta, por tanto, la consistencia eventual: durante un breve intervalo, una póliza recién emitida puede no tener todavía su plan de cobro.

Contratos publicados, no modelos. Los mensajes transportan identificadores y solo los datos que cada consumidor necesita, en un formato publicado. Nunca transportan objetos del modelo interno: cada consumidor traduce el mensaje a su propio modelo (mecanismo M5 de la Sección 3.2).

3.4. Relación con Onion/Clean y el Adaptador de Servicio

Los contextos delimitados organizan el sistema a gran escala. Internamente, cada contexto se construye según la Arquitectura Cebolla (Onion) o la Arquitectura Limpia (Clean). El diseño se apoya en tres principios. Los dos primeros son propios de estos estilos; el tercero lo formula la Guía de Arquitectura N-Capas:

- **El dominio en el centro.** El modelo de dominio ocupa el centro y todo el acoplamiento apunta hacia él. Las interfaces se definen en el núcleo y las implementaciones que dependen de tecnologías concretas viven en los bordes (Palermo, 2008a).
- **La Regla de Dependencia.** Las dependencias del código fuente solo pueden apuntar hacia adentro, hacia las políticas de más alto nivel (Martin, 2017, p. 203).
- **El dominio desacoplado y una Capa de Aplicación delgada.** La Guía insiste en desacoplar la capa de Dominio mediante contratos, interfaces, inyección de dependencias e inversión de control, para que las reglas de negocio queden aisladas y soporten un ritmo alto de cambios (De la Torre Llorente et al., 2010, p. 159). La Capa de Aplicación debe ser delgada: coordina las tareas y las conversiones de formato, pero no contiene lógica de negocio (De la Torre Llorente et al., 2010, p. 247). Interpretamos que, en este pasaje, la Guía llama «lógica de negocio» a la lógica del dominio. En la terminología de Martin, los casos de uso sí contienen reglas de negocio propias de la aplicación, mientras que las reglas críticas residen en las entidades (Martin, 2017, p. 204).

La Figura 3.2 aplica esta estructura al contexto de Siniestros.

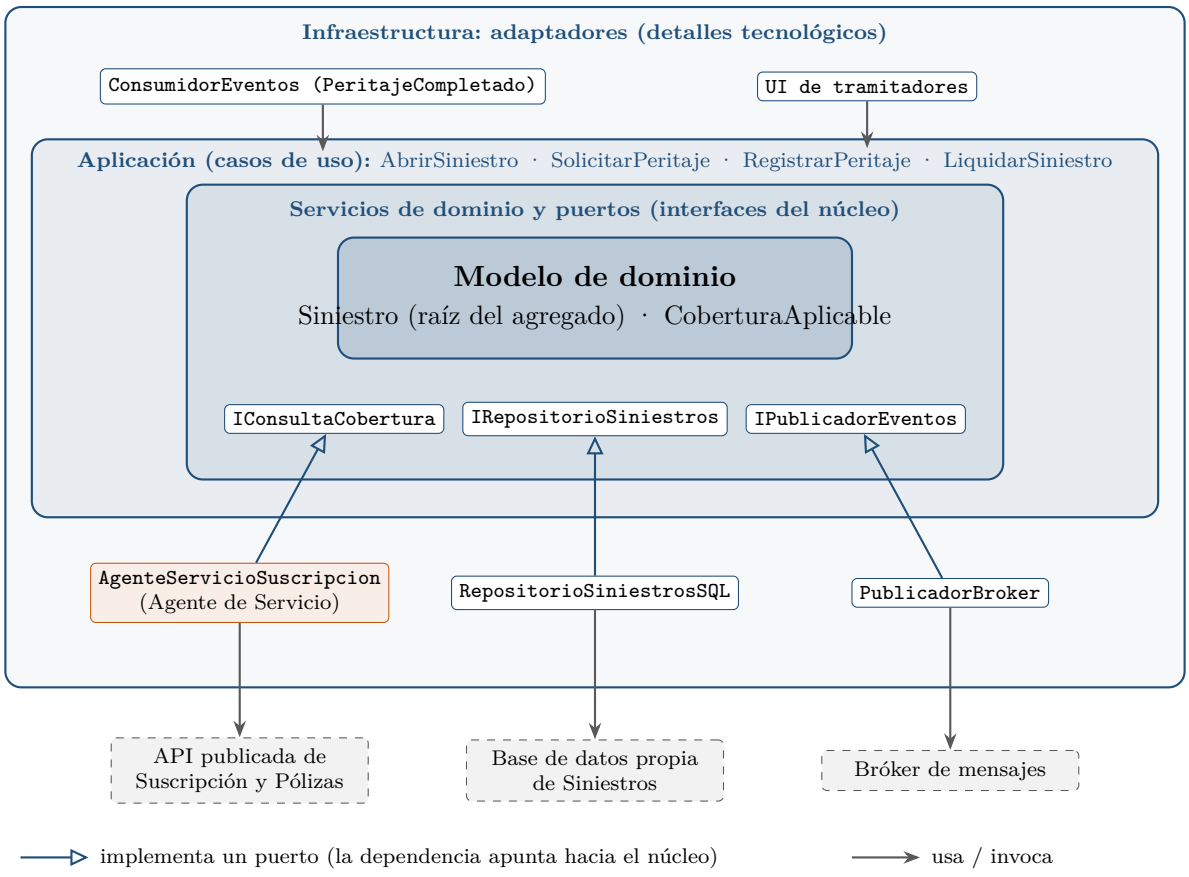


Figura 3.2: Estructura interna Onion/Clean del contexto de Siniestros. El Agente de Servicio es el único componente que conoce el protocolo y el formato de Suscripción.

El Adaptador de Servicio. Lo que el enunciado llama «Adaptador de Servicio» corresponde en la Guía a los *Agentes de Servicios externos*. La Guía emplea además «Agente de Servicio» en otro sentido, para un componente de la capa de presentación que consume servicios distribuidos (De la Torre Llorente et al., 2010, p. 269); en esta respuesta, «Agente de Servicio» designa siempre a un agente de servicios externos. Según la Guía, estos agentes:

- gestionan la semántica de comunicación con un servicio externo y mapean entre su formato de datos y el de la aplicación (De la Torre Llorente et al., 2010, p. 108);
- aíslan las particularidades de ese servicio, de modo que, manteniendo ciertas interfaces, el servicio puede sustituirse por otro sin afectar al sistema (De la Torre Llorente et al., 2010, p. 58);
- en aplicaciones grandes actúan como nivel de abstracción entre la capa de Dominio y los servicios remotos, con una interfaz homogénea. Conviene desacoplarlos mediante inversión de control, lo que permite simular los servicios externos con *fakes* durante el desarrollo y probar los propios agentes de forma unitaria (De la Torre Llorente et al., 2010, pp. 155–156).

Listado 3.1: Esbozo en estilo C# del puerto y del Agente de Servicio en el contexto de Siniestros.

```
// --- Núcleo de Siniestros: el puerto lo define el dominio ---
public interface IConsultaCobertura {
    CoberturaAplicable ObtenerCobertura(string numeroPoliza, DateTime fechaSiniestro);
}

// Objeto de valor propio de Siniestros: NO es la Póliza de Suscripción
public record CoberturaAplicable(string NumeroPoliza, bool Vigente,
                                 decimal LimiteIndemnizacion, decimal Franquicia);

// --- Infraestructura de Siniestros: Agente de Servicio (adaptador) ---
public class AgenteServicioSuscripcion : IConsultaCobertura {
    private readonly IClienteHttp http;   // el detalle tecnológico queda confinado aqu
    í

    public AgenteServicioSuscripcion(IClienteHttp http) { this.http = http; }

    public CoberturaAplicable ObtenerCobertura(string numeroPoliza, DateTime fecha) {
        // Contrato publicado por Suscripción (formato externo)
        string ruta = $"/polizas/{numeroPoliza}/cobertura?fecha={fecha:yyyy-MM-dd}";
        CoberturaDto dto = http.Get<CoberturaDto>(ruta);
        // Traducción al modelo de Siniestros: lo externo no entra al núcleo
        return new CoberturaAplicable(dto.Numero, dto.Estado == "VIGENTE",
                                      dto.LimiteDanos, dto.Franquicia);
    }
}
```

Es la misma idea que el adaptador de la Arquitectura Hexagonal (Cockburn, 2005) y que la capa de adaptadores de interfaz de la Arquitectura Limpia, que incluye los adaptadores hacia servicios externos (Martin, 2017, p. 205). En el diseño propuesto, `AgenteServicioSuscripcion` implementa el puerto `IConsultaCobertura`, definido en el núcleo de Siniestros. Llama a la API publicada de Suscripción y traduce la respuesta al objeto de valor local `CoberturaAplicable` (Listado 3.1).

El Agente de Servicio cierra el círculo de la Sección 3.2. Es la barrera física que impide que el modelo de Póliza de Suscripción entre en el núcleo de Siniestros, y aporta tres beneficios:

- un cambio en la API o en el formato de Suscripción solo afecta a este adaptador;
- el dominio de Siniestros puede probarse de forma aislada con una implementación simulada del puerto, del mismo modo que la Arquitectura Hexagonal permite desarrollar y probar la aplicación aislada de sus dispositivos y bases de datos (Cockburn, 2005);
- como el núcleo solo conoce la interfaz `IConsultaCobertura`, se respeta la Regla de Dependencia (Martin, 2017, p. 203).

Los consumidores de eventos cumplen el mismo papel de traducción en sentido de entrada: convierten cada mensaje publicado en una llamada a un caso de uso del contexto receptor.

Valoración crítica

La solución reúne las ventajas que justifican DDD en un dominio con reglas de negocio cambiantes y varias áreas de expertos:

- cada modelo evoluciona al ritmo de su área;
- los cambios quedan contenidos dentro de un contexto;
- el lenguaje del código coincide con el de los expertos;
- cada núcleo de dominio puede probarse sin infraestructura.

Tiene, sin embargo, costes que deben asumirse de forma consciente:

(1) **Duplicación de datos y de código entre contextos**, que es el precio del desacoplamiento (Richards y Ford, 2020, cap. 17).

(2) **Consistencia eventual y menor control del flujo.** Los contextos pueden estar momentáneamente desalineados. Además, la topología *broker* complica el control del flujo, la gestión de errores y la recuperación, porque ningún componente supervisa el proceso completo (Richards y Ford, 2020, cap. 14).

(3) **Dependencia temporal en la consulta síncrona.** Si Suscripción no está disponible, Siniestros no puede verificar la cobertura al abrir un expediente. Es el mismo riesgo por el que se evitó acoplar de forma síncrona a Peritaje (criterio C4). Como mitigación, propia de este diseño, Siniestros puede registrar el expediente en estado «cobertura pendiente de verificar» y completar la verificación cuando Suscripción responda, sin bloquear la atención al asegurado. Para ello, el puerto `IConsultaCobertura` debe contemplar el fallo de la llamada; el Listado 3.1 omite ese manejo por brevedad.

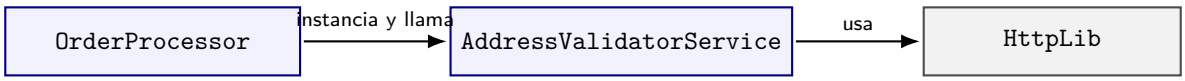
(4) **Más infraestructura y riesgo de granularidad excesiva.** Hay que operar un bróker de mensajes. Además, el error frecuente de crear servicios demasiado pequeños obliga después a reconstruir enlaces de comunicación entre ellos (Richards y Ford, 2020, cap. 17).

La propia Guía advierte que, si una aplicación es sencilla, sus reglas de negocio cambiarán poco y no se prevén cambios de infraestructura, una arquitectura orientada al dominio probablemente no compensa (De la Torre Llorente et al., 2010, p. 42). No es el caso de la aseguradora: cuatro áreas con lenguajes propios y reglas contractuales, de cobro y de peritaje que evolucionan de forma independiente son justamente el escenario en el que la división en contextos delimitados, comunicados por eventos y protegidos por Agentes de Servicios externos, aporta más valor que coste.

P4: Evaluación de la Inversión de Dependencias en la Práctica: OrderProcessor

4.1. a) ¿Por qué viola la dependencia directa los principios de Hexagonal, Limpia y Onion?

Un equipo de desarrollo trabaja en un sistema de gestión de pedidos. La lógica de negocio principal reside en la clase `OrderProcessor`, que necesita comunicarse con un servicio externo de validación de direcciones. La implementación actual tiene una dependencia directa:



Esta arquitectura viola los principios fundamentales de las arquitecturas Hexagonal, Limpia y Onion por las siguientes razones:

Violación del Principio de Inversión de Dependencias (DIP)

El DIP establece que «las dependencias del código fuente deben referirse solo a abstracciones, no a concreciones» (Martin, 2017, p. 87). En la implementación actual:

1. `OrderProcessor`, la política de alto nivel, depende directamente de un detalle de implementación concreto y volátil: `AddressValidatorService`.
2. El servicio externo es inherentemente volátil: puede cambiar su API, su contrato, su disponibilidad, o ser reemplazado por otro proveedor.
3. Como señala Martin (2017, p. 88): «Every change to an abstract interface corresponds to a change to its concrete implementations. Conversely, changes to concrete implementations do not always, or even usually, require changes to the interfaces that they implement. Therefore interfaces are less volatile than implementations.»

Violación por arquitectura

- **Arquitectura Hexagonal:** La lógica de negocio (interior del hexágono) depende de un servicio externo (adaptador), invirtiendo la relación correcta. Los puertos (interfaces) deben definirse en el interior y los adaptadores (implementaciones) en el exterior; los adaptadores dependen de los puertos, no al revés (Cockburn, 2005, «Nature of the Solution»).

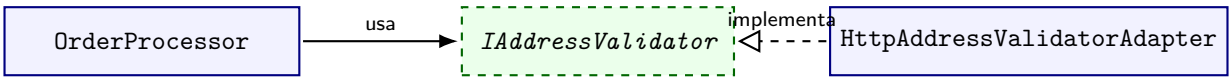
- **Arquitectura Limpia:** Se viola la Regla de Dependencia: «Source code dependencies must point only inward, toward higher-level policies. Nothing in an inner circle can know anything at all about something in an outer circle» (Martin, 2017, p. 203). Las dependencias apuntan hacia afuera (hacia detalles volátiles) en lugar de hacia adentro (hacia políticas de alto nivel).
- **Arquitectura de Cebolla:** La dependencia se aleja del centro (Modelo de Dominio). En Onion todas las dependencias deben apuntar hacia el centro, y las capas externas deben implementar interfaces definidas en las capas internas (Martin, 2017, p. 202).

Acoplamiento transitivo a la librería HTTP

La cadena `OrderProcessor` → `AddressValidatorService` → `HttpClient` acopla transitivamente la lógica de negocio con una librería HTTP concreta. Cualquier cambio en la API del servicio, su contrato o la librería HTTP obliga a modificar `OrderProcessor`, la clase que aloja la política de negocio principal.

Dirección correcta de las dependencias

El DIP y la Regla de Dependencia exigen que las dependencias del código fuente apunten hacia las abstracciones del núcleo:



La política de alto nivel y el detalle dependen ahora de la misma abstracción, definida en el núcleo: `OrderProcessor` usa `IAddressValidator` y `AddressValidatorService` la implementa. Nótese que el *flujo de control* se mantiene —`OrderProcessor` sigue solicitando la validación—; lo que se invierte es la *dependencia del código fuente*, que ahora apunta desde el detalle hacia la abstracción (Martin, 2017, pp. 87, 91).

4.2. b) Rediseño con inversión de dependencias

Propuesta de solución

Para alinear el diseño con los principios de Clean Architecture, Arquitectura Hexagonal y Arquitectura de Cebolla —arquitecturas cuya característica común es producir sistemas independientes de frameworks, UI, base de datos y agentes externos (Martin, 2017, p. 202), regidas por la Regla de Dependencia (Martin, 2017, p. 203)— se realizan tres cambios:

1. **Crear una abstracción en el núcleo (Capa de Dominio/Entidades):**
   - Definir la interfaz `IAddressValidator` (puerto de validación) en la capa más interna.
   - Define el contrato: `validarDireccion(Direccion): ResultadoValidacion`.

- La interfaz es estable y poco volátil: «Don't refer to volatile concrete classes. Refer to abstract interfaces instead» (Martin, 2017, p. 89).

2. **Implementar la interfaz en la capa de infraestructura (exterior del hexágono):**
   - `AddressValidatorService` implementa `IAddressValidator`.
   - Reside en la capa de infraestructura (adaptador externo en Hexagonal).
   - Encapsula la dependencia de la librería HTTP concreta.
3. **Inyectar la dependencia en `OrderProcessor`:**
   - `OrderProcessor` recibe `IAddressValidator` por constructor (inyección de dependencias).
   - `OrderProcessor` **nunca** instancia directamente `AddressValidatorService`.
   - Se usa *Abstract Factory* o contenedor de DI para la creación (Martin, 2017, p. 89).

Diagrama de dependencias antes/después

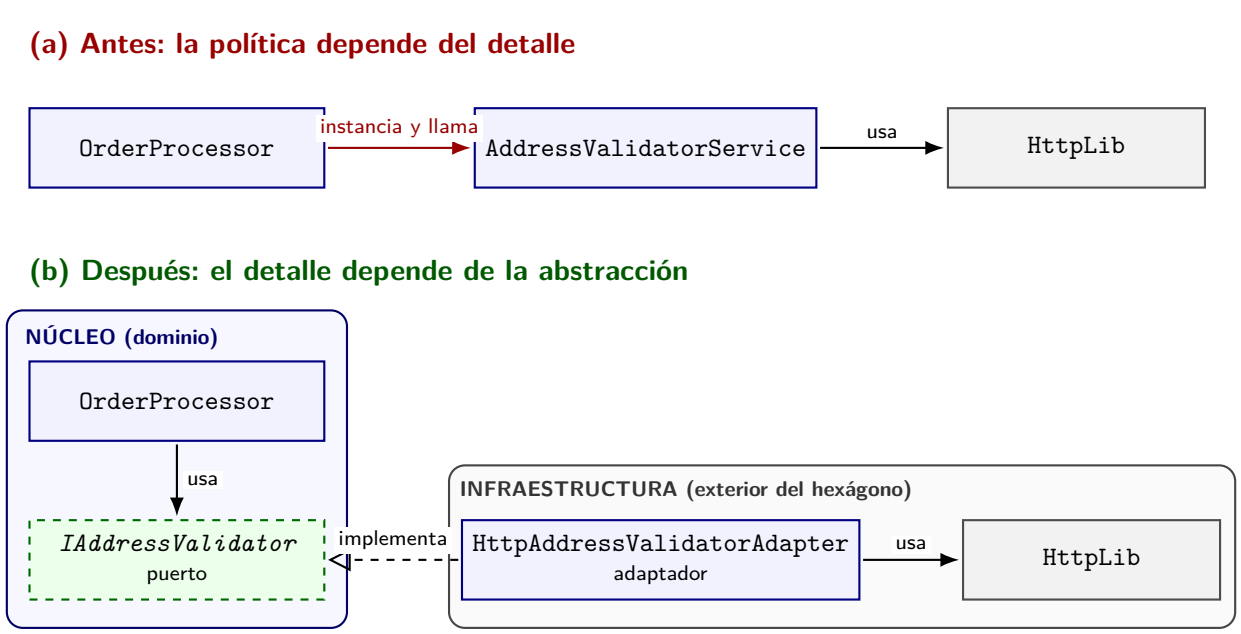


Figura 4.1: Dependencias del código fuente antes y después de la inversión. En (a) la lógica de negocio depende de un detalle concreto; en (b) tanto `OrderProcessor` como el adaptador dependen de la interfaz definida en el núcleo, y la única dependencia que cruza la frontera apunta hacia dentro. `HttpAddressValidatorAdapter` es el `AddressValidatorService` original rediseñado como adaptador.

Justificación teórica

- **Regla de Dependencia** (Martin, 2017, p. 203): Las dependencias apuntan hacia adentro; `AddressValidatorService` (infraestructura) depende de `IAddressValidator` (núcleo).

- **Independencia de frameworks** (Martin, 2017, p. 202): La arquitectura no depende de librerías HTTP; se usan como herramientas intercambiables.
- **Testabilidad** (Martin, 2017, p. 202): El núcleo se prueba sin UI, BD, servidor web ni servicios externos.
- **Propiedad del contrato**: `OrderProcessor` define *qué* necesita (el contrato `IAddressValidator`); la infraestructura provee *cómo* se cumple. El detalle depende de la abstracción del núcleo.

4.3. c) Pruebas aisladas sin HTTP y mantenibilidad al cambiar de proveedor

Aislamiento de la lógica de negocio

Tras el rediseño, `OrderProcessor` no se encarga de la validación de direcciones ni de detalles de comunicación HTTP. Su única responsabilidad es la lógica de negocio de procesamiento de pedidos. La validación se delega a la implementación inyectada de `IAddressValidator`.

Pruebas unitarias puras con test doubles

Ahora `OrderProcessor` es testeable de forma aislada, rápida y determinista:

1. **Mock/Stub de `IAddressValidator`:** En las pruebas unitarias se inyecta un *test double* que implementa `IAddressValidator` y devuelve respuestas controladas (válida, inválida, error de red, timeout, etc.).
2. **Sin dependencias externas:** No se requiere conexión a red, servicio externo real, ni base de datos.
3. **Cobertura de casos frontera:** Fácil simulación de errores de validación, timeouts, respuestas malformadas.
4. **Velocidad:** Las pruebas se ejecutan en milisegundos, permitiendo integración continua rápida.

Implicaciones para la mantenibilidad y el cambio de proveedor

El diseño refactorizado confina la volatilidad a un único punto del sistema:

- **Cambio de proveedor:** Sustituir el proveedor de validación solo requiere escribir un nuevo adaptador que implemente `IAddressValidator` y reconfigurar el mecanismo de construcción (Abstract Factory o contenedor DI); `OrderProcessor` y el resto del núcleo permanecen intactos.

- **Volatilidad contenida:** Dado que las interfaces son menos volátiles que las implementaciones (Martin, 2017, p. 88), el núcleo depende del elemento más estable; la volatilidad del proveedor queda confinada a la capa de infraestructura.
- **Extensión sin modificación:** Agregar un segundo proveedor (redundancia, tolerancia a fallos) no exige modificar el núcleo, solo sumar un nuevo adaptador que implemente la misma interfaz.
- **Impacto localizado de cambios:** Un cambio de contrato (interfaz) obliga a actualizar núcleo y adaptadores y se detecta en tiempo de compilación; un cambio de implementación no propaga modificaciones hacia el núcleo.

Principio subyacente

Como establece Martin (2017, p. 202), una característica fundamental de estas arquitecturas es ser **Testable**: «The business rules can be tested without the UI, database, web server, or any other external element.» La inversión de dependencia es el mecanismo que hace esto posible.

P5: Análisis del Kitty Problem y el Principio Abierto/Cerrado

5.1. Por qué la división por verbos o entidades provocó el *Kitty Problem*

El *Kitty Problem* muestra cómo un requisito transversal, la entrega de gatitos, obliga a cambiar de forma coordinada todos los servicios de un sistema de taxis. Aunque el enunciado sitúa el ejemplo en *Fundamentals of Software Architecture* (Richards y Ford, 2020), en la bibliografía consultada el caso aparece en Martin (2017, cap. 27), que es la fuente que se cita en esta respuesta.

Martin parte de un agregador de taxis, ya presentado en el capítulo sobre el Principio de Sustitución de Liskov, que conoce a muchos proveedores de taxis de una ciudad y permite a los clientes pedir viajes (Martin, 2017, pp. 80, 242). Para que fuera escalable, el sistema se construyó con muchos microservicios, y el personal se dividió en equipos pequeños, cada uno responsable de unos pocos servicios (Martin, 2017, p. 242):

- `TaxiUI` atiende a los clientes, que piden taxis desde el móvil;
- `TaxiFinder` consulta los inventarios de los proveedores y determina los taxis candidatos;
- `TaxiSelector` elige uno según los criterios del cliente (coste, tiempo, lujo);
- `TaxiDispatcher` encarga el taxi elegido.

Tras más de un año en producción, el departamento de *marketing* anuncia un servicio de entrega de gatitos. Habrá puntos de recogida en la ciudad; un taxi cercano recogerá el gatito y lo llevará a la dirección indicada. Por ahora, un proveedor ha aceptado participar; es probable que algunos se sumen y otros lo rechacen. Los conductores alérgicos a los gatos nunca deben ser elegidos, y un vehículo que haya transportado gatitos en los últimos tres días no debe asignarse a clientes que declaren esa alergia (Martin, 2017, p. 243). La consecuencia es que *todos* los servicios tienen que cambiar y su desarrollo y despliegue deben coordinarse: los servicios están acoplados y no pueden desarrollarse, desplegarse ni mantenerse de forma independiente (Martin, 2017, p. 243). La Figura 5.1 resume la situación.

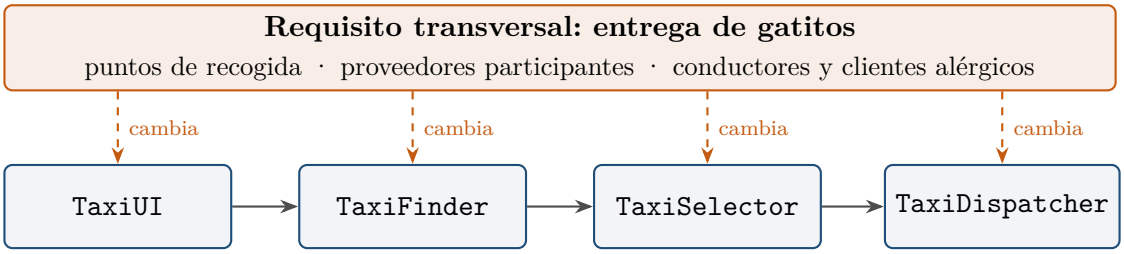


Figura 5.1: El *Kitty Problem*: el nuevo requisito atraviesa los cuatro servicios, que deben cambiar y desplegarse de forma coordinada. Elaboración propia a partir de Martin (2017, pp. 242–243).

La descomposición funcional es vulnerable a los cambios transversales. Cada servicio implementa un paso del flujo: encontrar, seleccionar, despachar. Un requisito que introduce un *tipo nuevo* de pedido no cae en un paso, sino en todos. Martin lo resume así: todo sistema se enfrenta a los aspectos transversales, y las descomposiciones funcionales como la de este ejemplo son muy vulnerables a las funcionalidades que atraviesan todos sus pasos (Martin, 2017, p. 243).

Los servicios no estaban realmente desacoplados. Separar en procesos distintos desacopla las variables, pero no los datos: si se añade un campo a un registro que circula entre servicios, todos los que lo usan deben cambiar y acordar su interpretación. En esa medida, el desarrollo, el despliegue y la operación deben coordinarse (Martin, 2017, pp. 240–241). En el ejemplo, la lista de candidatos que `TaxiFinder` deja para `TaxiSelector` tendría que incorporar la información sobre gatitos y alergias.

Las fronteras siguen los pasos del flujo, no el dominio. Richards y Ford (2020, cap. 8) distinguen la partición técnica, por capas como presentación, reglas de negocio o persistencia, de la partición por dominio. Con la primera, los flujos de negocio, que en la mayoría de los sistemas reales atraviesan las capacidades técnicas, quedan repartidos por todas las capas; con la segunda, los componentes de nivel superior se organizan por dominio o por flujo completo, lo que refleja mejor los cambios que suelen ocurrir. Los pasos del flujo no son capas técnicas en sentido estricto, pero el efecto es análogo: la nueva capacidad de negocio queda repartida entre

todos ellos. Dividir por entidades tampoco lo resolvería: con un servicio por entidad (taxis, pedidos, proveedores), la entrega de gatitos seguiría afectando a todos (inferencia propia). Además, esa división cae en el antipatrón *entity trap*, que confunde las relaciones entre datos con los flujos de trabajo de la aplicación (Richards y Ford, 2020, cap. 8).

Se violó el Principio de Cierre Común (CCP). El CCP pide reunir en un mismo componente lo que cambia por las mismas razones y al mismo tiempo, de modo que un cambio de requisitos afecte al menor número posible de componentes (Martin, 2017, pp. 105–106). Todas las reglas de los gatitos cambian juntas, pero el diseño las dispersa por cuatro servicios. Martin vincula expresamente este principio con el *Kitty Problem* (Martin, 2017, p. 106).

5.2. Diseño con DDD y Onion/Clean: un solo componente afectado

La clave es trazar las fronteras sobre un *eje de cambio*: los componentes de cada lado de una frontera cambian a ritmos distintos y por razones distintas (Martin, 2017, p. 173). En este sistema, la propuesta toma como eje el *tipo de servicio de transporte* (pasajeros, gatitos y los que se añadan en el futuro) en lugar de los pasos del flujo; la Sección 5.3 discute el coste de esa elección. El diseño se construye sobre tres ideas.

1. Un núcleo de políticas estables, protegido por la Regla de Dependencia. Las políticas genéricas de encontrar, seleccionar y despachar un vehículo forman el núcleo, expresado con clases abstractas e interfaces. Las dependencias del código fuente solo pueden apuntar hacia adentro, hacia las políticas de más alto nivel (Martin, 2017, p. 203). Si el núcleo debe quedar protegido de los cambios en las variantes, son las variantes las que deben depender del núcleo (Martin, 2017, pp. 73–74). En términos de la Arquitectura Onion, el diseño se distribuye en anillos: todo acoplamiento apunta hacia el centro, las interfaces se definen en el núcleo y las implementaciones que dependen de detalles viven en los bordes (Palermo, 2008a). La correspondencia, que es una interpretación propia, es la siguiente:

- **centro:** las políticas de despacho y sus abstracciones (el selector `Selector`, la interfaz de reglas `IReglaSeleccion` y el puerto hacia los proveedores, `IProveedorTaxis`);
- **anillo intermedio:** `Rides` y `Kittens`, reglas de negocio de cada tipo de servicio (no detalles tecnológicos) que dependen del centro y lo extienden;
- **anillo exterior:** los adaptadores hacia los sistemas externos, que constituyen la capa de adaptadores de interfaz (Martin, 2017, p. 205), y la interfaz de usuario con `Main`, el componente que configura el sistema (Martin, 2017, p. 237).

El criterio que ordena los anillos es el nivel: lo de más alto nivel queda en el centro, protegido de los cambios en lo de nivel inferior (Martin, 2017, pp. 73–74). Por eso, a diferencia del esquema de Palermo (2008a), que sitúa el modelo de dominio en el centro, aquí las reglas de cada tipo de servicio quedan fuera del núcleo: son de nivel inferior a las políticas genéricas de despacho y son las que más varían.

2. Contextos por capacidad de negocio (DDD). Cada tipo de servicio se convierte en un componente que agrupa su modelo: `Rides` para los viajes de pasajeros y `Kittens` para las entregas de gatitos. Desde DDD, cada uno se comporta como un Contexto Delimitado: lo relativo a su dominio es visible dentro y opaco fuera, y cada contexto define sus propias entidades en lugar de compartir una clase única (Richards y Ford, 2020, cap. 7). El modelo de cada contexto se expresa en un Lenguaje Ubicuo, el lenguaje de los expertos del dominio (De la Torre Llorente et al., 2010, p. 21); que cada contexto tenga el suyo es una aplicación propia de esa idea:

- en `Rides`, el cliente es un *pasajero* y el pedido es un `Viaje`;
- en `Kittens`, el cliente es quien recibe el gatito y el pedido es una `Entrega` con un punto de recogida.

La delimitación no es total, y conviene decirlo. Ambos contextos comparten a propósito con el núcleo un modelo mínimo: los identificadores (vehículo, conductor, cliente) y las abstracciones del despacho, que `Viaje` y `Entrega` extienden. Es un acoplamiento aceptado: cada contexto está delimitado en su modelo propio, pero no es autónomo. Las alergias y el historial de entregas, en cambio, solo existen en el lenguaje de `Kittens` y viven en su modelo, no en el núcleo. La *Guía de Arquitectura N-Capas* propone una organización análoga para el modelo de dominio: módulos verticales por área de negocio, de alta cohesión y bajo acoplamiento, que se comunican mediante interfaces y cuyos nombres forman parte del Lenguaje Ubicuo (De la Torre Llorente et al., 2010, pp. 71–72). El resultado es una partición por dominio en la que cada capacidad es funcionalmente cohesiva (Richards y Ford, 2020, caps. 8 y 17), y que cumple el CCP, porque lo que cambia junto vive junto (Martin, 2017, p. 105).

3. Las fronteras atraviesan los servicios. Los servicios no son lo que define la arquitectura. Para tratar los aspectos transversales, cada servicio debe tener una arquitectura interna de componentes que siga la Regla de Dependencia: las fronteras arquitectónicas no caen entre servicios, sino que los atraviesan (Martin, 2017, pp. 246–247). En el despliegue, los servicios pueden seguir existiendo como antes: cada uno tiene su diseño interno de componentes, y la nueva funcionalidad se incorpora como clases derivadas, alojadas en sus propios componentes, que se añaden a la ruta de carga de cada servicio (Martin, 2017, p. 246). La Figura 5.2 muestra, por tanto, una vista lógica. `Kittens` es un componente lógico que puede desplegarse como varios archivos *jar*, uno en cada servicio que lo necesite. El historial de entregas también pertenece a `Kittens`: lo actualiza su parte de despacho y lo consulta su regla de selección. Si esas partes se despliegan en servicios distintos, el historial debe residir en un almacén propio de `Kittens` al que ambas accedan.

Con este diseño, incorporar la entrega de gatitos consiste en añadir el componente `Kittens` y ajustar la interfaz de usuario y su configuración. El núcleo y `Rides` no se modifican. Hay que reconocer que, si `Kittens` se reparte en varios servicios, la incorporación de sus archivos *jar* en ellos debe coordinarse. La diferencia con el diseño original es que el cambio es aditivo: no se modifica ni se recompila ningún componente del núcleo ni de `Rides`, y la funcionalidad queda confinada en un único componente lógico (aunque no en un único servicio), más un ajuste en la capa externa. Conviene precisar que, incluso en la solución de Martin, `TaxiUI` cambia (Martin, 2017, p. 244): el cambio no afecta *literalmente* a un único componente. Es

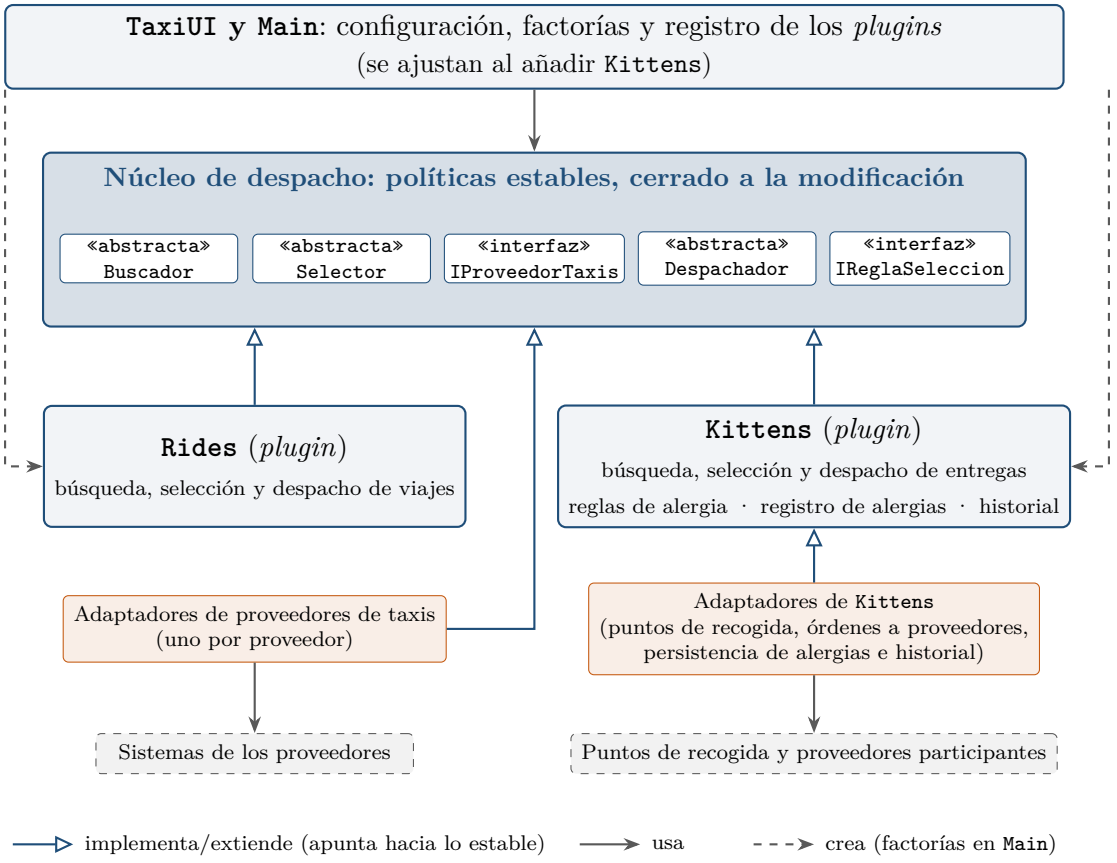


Figura 5.2: Solución propuesta (vista lógica): `Rides` y `Kittens` se enchufan al núcleo como *plugins*, y los adaptadores aíslan los sistemas externos. Entre núcleo y *plugins*, las dependencias apuntan hacia el núcleo; solo `TaxiUI` y `Main`, en la capa externa, conocen los *plugins* concretos. Elaboración propia a partir de Martin (2017, pp. 205, 232, 237, 244–246), Cockburn (2005) y Richards (2015, pp. 21–23).

lo esperable: el cierre total es inalcanzable y, por tanto, debe ser estratégico (Martin, 2017, p. 106).

5.3. Solución con `Kittens` y `Rides` como *plugins*

La solución de Martin. Al aplicar SOLID, Martin conserva gran parte de la lógica de los servicios originales en clases base abstractas. Lo específico de los viajes se extrae a un componente `Rides`, y la nueva capacidad se sitúa en un componente `Kittens`. Ambos extienden las clases abstractas mediante *Template Method* o *Strategy*, siguen la Regla de Dependencia y son creados por factorías bajo el control de la interfaz de usuario. Al implementar la entrega de gatitos, cambia `TaxiUI`, pero nada más: se añade un nuevo *jar*, *Gem* o DLL que se carga dinámicamente en tiempo de ejecución, de modo que la extensión queda desacoplada y puede desarrollarse y desplegarse de forma independiente (Martin, 2017, p. 244). En la propuesta, esa independencia lo es respecto del núcleo y de `Rides`; entre las piezas de `Kittens` repartidas en varios servicios, el despliegue se coordina, y la capa externa se ajusta (Sección 5.2). La

Figura 5.2 muestra la estructura propuesta.

Por qué respeta el OCP. El OCP, formulado originalmente por Meyer en 1988 y citado aquí a través de Martin, establece que un artefacto de software debe estar abierto a la extensión y cerrado a la modificación (Martin, 2017, p. 70). En el plano arquitectónico se consigue separando la funcionalidad según cómo, por qué y cuándo cambia, y organizándola en una jerarquía de componentes en la que los de nivel superior quedan protegidos de los cambios en los de nivel inferior (Martin, 2017, p. 74). En la propuesta:

- el núcleo está *cerrado*: añadir `Kittens` no modifica ninguna de sus clases;
- el sistema está *abierto*: la nueva capacidad entra como un componente nuevo que extiende las abstracciones del núcleo. Martin lo plantea en términos de despliegue: una nueva funcionalidad es un nuevo *jar* que extiende las clases abstractas y se añade a la ruta de carga del servicio, lo que se ajusta al OCP (Martin, 2017, p. 246).

Esta estructura coincide con el patrón *microkernel*, también llamado de *plugins*: un sistema núcleo al que se añaden módulos *plugin* independientes que aportan nuevas capacidades de negocio, y que el núcleo suele localizar mediante un registro (Richards, 2015, pp. 21–22). En la propuesta, `Main` sustituye a ese registro: en lugar de que el núcleo localice los *plugins*, `Main` los crea y le entrega sus reglas por inyección de dependencias, tarea que Martin (2017, p. 232) sitúa precisamente en ese componente, al que describe como el más «sucio» de todos. Además, `Main` es en sí mismo un *plugin* de la aplicación, y puede haber varios, uno por configuración (por ejemplo, por país, jurisdicción o cliente) (Martin, 2017, p. 237). Aplicado a este caso, eso permitiría activar `Kittens` solo donde se ofrezca la entrega de gatitos.

Ejemplo: las reglas de alergia como reglas de selección enchufables. El requisito de las alergias muestra hasta dónde llega la extensión (Listado 5.1). El `Selector` del núcleo es un *Template Method*: fija el algoritmo y deja la ordenación a cada *plugin*. Además, recibe las reglas registradas, que funcionan como filtros que combina sin conocerlas. Las reglas solo usan identificadores que el núcleo ya exponía (de vehículo, de conductor y de cliente). `Kittens` aporta dos reglas:

- excluir a los conductores alérgicos en las entregas;
- no asignar a un cliente alérgico un vehículo que haya llevado gatitos en los últimos tres días.

La segunda regla afecta también a los viajes de pasajeros y, aun así, ni `Rides` ni las entidades del núcleo se modifican: las alergias y el historial de entregas pertenecen al modelo de `Kittens`, que los consulta por identificador. Que el núcleo acepte reglas adicionales es la decisión de cierre estratégico que hace posible la extensión (criterio propio a partir de Martin, 2017, p. 106).

Listado 5.1: Esbozo en estilo C# del punto de extensión del núcleo y de las reglas que aporta el *plugin* `Kittens`.

```
// --- Núcleo de despacho (cerrado a la modificación) ---
// Modelo mínimo compartido: solo identificadores.
public abstract class Pedido { public string ClienteId { get; init; } }
public class Vehiculo {
    public string Id { get; init; }
    public string ConductorId { get; init; }
}

public interface IReglaSeleccion {
    bool Admite(Vehiculo vehiculo, Pedido pedido);
}

// Template Method: el algoritmo es fijo; ordenar es del plugin.
public abstract class Selector {
    // Reglas aportadas por los plugins, inyectadas por Main.
    private readonly IEnumerable<IReglaSeleccion> reglas;
    protected Selector(IEnumerable<IReglaSeleccion> r) { reglas = r; }

    // Devuelve null si ningún candidato supera las reglas.
    public Vehiculo? Elegir(IEnumerable<Vehiculo> c, Pedido p) {
        var admitidos = c.Where(v => reglas.All(r => r.Admite(v, p)));
        return Ordenar(admitidos, p).FirstOrDefault();
    }
    protected abstract IEnumerable<Vehiculo> Ordenar(
        IEnumerable<Vehiculo> candidatos, Pedido pedido);
}

// --- Plugin Kittens: modelo y reglas propios ---
public class Entrega : Pedido { /* punto de recogida, destino */ }

// Datos de Kittens, consultados por identificador.
public interface IRegistroAlergias {
    bool ConductorAlergico(string conductorId);
    bool ClienteAlergico(string clienteId);
}
public interface IHistorialEntregasGatitos {
    bool HuboEntregaEnLosUltimosDias(string vehiculoId, int dias);
}

public class ReglaConductorNoAlergico : IReglaSeleccion {
    private readonly IRegistroAlergias alergias;
    public ReglaConductorNoAlergico(IRegistroAlergias a) {
        alergias = a;
    }
    public bool Admite(Vehiculo v, Pedido p) =>
        p is not Entrega || !alergias.ConductorAlergico(v.ConductorId);
}

public class ReglaClienteAlergico : IReglaSeleccion {
    private readonly IRegistroAlergias alergias;
    private readonly IHistorialEntregasGatitos historial;
```

```
    public ReglaClienteAlergico(IRegistroAlergias a,
                                IHistorialEntregasGatitos h) {
        alergias = a; historial = h;
    }
    public bool Admite(Vehiculo v, Pedido p) =>
        !alergias.ClienteAlergico(p.ClienteId)
        || !historial.HuboEntregaEnLosUltimosDias(v.Id, 3);
}
```

El papel de los adaptadores. El capítulo de Martin sobre el *Kitty Problem* no menciona los adaptadores. Su papel en esta extensión se argumenta, como interpretación propia, a partir de la bibliografía:

- **Hacia los sistemas externos.** Un adaptador específico de cada tecnología traduce entre el mundo exterior y un puerto de la aplicación; la aplicación desconoce lo que hay al otro lado (Cockburn, 2005, «Intent»). En la Arquitectura Limpia, los adaptadores forman la capa de adaptadores de interfaz, que convierte los datos del formato de los casos de uso y las entidades al de los agentes externos, como otros servicios (Martin, 2017, p. 205). La *Guía de Arquitectura N-Capas* los llama agentes de servicios externos: aíslan las particularidades de cada servicio, de modo que pueda sustituirse sin afectar al sistema (De la Torre Llorente et al., 2010, pp. 58, 108).
- **Adaptadores propios de `Kittens`.** El *plugin* necesita conocer los puntos de recogida, enviar las órdenes de recogida a los proveedores que participan y guardar sus propios datos. Lo hace mediante adaptadores que se añaden junto al *plugin*: uno por sistema externo y otro de persistencia, que implementa `IRegistroAlergias` e `IHistorialEntregasGatitos`. La alergia que declara un pasajero la registra `TaxiUI` en `Kittens`, no en el núcleo. Los adaptadores de proveedores que ya usa el núcleo no se modifican, porque `Kittens` no amplía los contratos del núcleo, sino que define otros nuevos (criterio propio). Así, ni el núcleo ni el *plugin* conocen formatos o protocolos externos.
- **Entre un *plugin* y el núcleo.** Cuando un *plugin* trae un contrato propio (por ejemplo, porque lo desarrolla un tercero), es habitual crear un adaptador entre ese contrato y el contrato estándar del núcleo (Richards, 2015, p. 23). Si la logística de gatitos la aportara una empresa asociada, un adaptador la conectaría al núcleo sin modificarlo.

En todos los casos, los adaptadores permiten que la extensión llegue hasta los sistemas reales sin romper la Regla de Dependencia: el detalle depende de la abstracción, nunca al revés.

Valoración crítica

El patrón de núcleo y *plugins* aporta lo que el *Kitty Problem* pedía:

- **agilidad alta**, porque los cambios quedan aislados en *plugins* débilmente acoplados y el núcleo se estabiliza pronto (Richards, 2015, p. 25);

- **facilidad de despliegue alta**: según la implementación, los *plugins* pueden añadirse en tiempo de ejecución (Richards, 2015, p. 25);
- **facilidad de prueba alta**, porque cada *plugin* puede probarse de forma aislada (Richards, 2015, p. 26).

Además, favorece el diseño evolutivo: se parte de un núcleo sólido y se añaden funcionalidades sin cambios importantes en él (Richards, 2015, pp. 24–25).

Tiene, sin embargo, costes y límites:

(1) **El eje de cambio es una apuesta.** El núcleo solo queda cerrado frente a los tipos de cambio más comunes que se esperan o que ya se han vivido (Martin, 2017, p. 106). Elegir el *tipo de servicio* como eje se justifica por un cambio concreto, y tiene contrapartida. Por ejemplo, un nuevo criterio de desempate común a todos los tipos de servicio (preferir el vehículo con menor tiempo estimado de llegada) obligaría a modificar el método `Ordenar` de cada *plugin* o a abrir el `Selector` del núcleo, y además a ampliar el modelo mínimo compartido con ese dato. El coste no es tanto la dispersión del cambio como la pérdida del cierre del núcleo, que es precisamente lo que el diseño quería proteger; en la división por pasos, ese criterio encajaría con naturalidad en `TaxiSelector`, que ya selecciona por tiempo (Martin, 2017, p. 242). Si un requisito futuro afecta a un eje que el núcleo no abstrajo, habrá que modificar el núcleo.

(2) **Acoplamiento entre *plugins*.** Los *plugins* deberían ser independientes entre sí y comunicarse lo mínimo (Richards, 2015, p. 22). La regla del cliente alérgico hace que `Kittens` influya en la selección de los viajes de `Rides`. El punto de extensión del núcleo vuelve explícito ese acoplamiento, pero no lo elimina: aunque cada *plugin* puede probarse aislado, la selección completa debe probarse con todas las reglas registradas.

(3) **Complejidad de desarrollo.** Exige un diseño cuidadoso y control de los contratos: versionado, registro de *plugins*, granularidad y formas de conexión (Richards, 2015, p. 26).

(4) **Escalabilidad.** Muchas implementaciones del patrón se despliegan como una sola unidad, con escalabilidad baja, aunque a veces puede lograrse por *plugin* (Richards, 2015, p. 26). Si se mantienen servicios distribuidos, hay que evitar el error de hacerlos demasiado pequeños (Richards y Ford, 2020, cap. 17).

En conclusión, el *Kitty Problem* no se debió a usar servicios, sino a trazar las fronteras de componentes sobre los pasos del flujo y no sobre el eje que más cambiaba. Un núcleo estable protegido por la Regla de Dependencia, capacidades de negocio cohesivas con su propio lenguaje (`Rides`, `Kittens`) que se enchufan como *plugins*, y adaptadores que aíslan lo externo convierten un cambio que afectaba a todo el sistema en una extensión localizada en un componente lógico, más un ajuste en la capa externa. Eso es lo que persigue el OCP.

P6: Evaluación de N-Capas frente a Atributos de Calidad: Open Layers y Sinkhole Anti-pattern

6.1. Capas abiertas, aislamiento de capas y mantenibilidad a largo plazo

El enunciado parte de una crítica conocida: la arquitectura N-Capas puntúa bajo en agilidad general, facilidad de despliegue y escalabilidad (Richards, 2015, pp. 8–9). Un estudiante sostiene que esas carencias pueden mitigarse con capas abiertas (*open layers*) y técnicas como el *fast-lane reader*. La afirmación es **tácticamente correcta pero estratégicamente insuficiente**. Correcta, porque ambos mecanismos son reales y están documentados: las propias fuentes los presentan para aliviar costes puntuales de la N-Capas (Richards y Ford, 2020, cap. 10). Insuficiente, porque ninguna de las dos técnicas corrige la causa estructural de esas calificaciones bajas —la naturaleza monolítica y el acoplamiento entre componentes (Richards, 2015, pp. 8–9)— y porque ambas se pagan con el único mecanismo que la N-Capas ofrecía para gestionar el cambio con seguridad: el Aislamiento de Capas.

En la N-Capas cada capa es **cerrada** o **abierta**. Una capa cerrada no puede saltarse: toda solicitud debe atravesar la capa inmediatamente inferior para llegar a la siguiente (Richards, 2015, p. 3); (Richards y Ford, 2020, cap. 10). Las capas cerradas son las que materializan el **Aislamiento de Capas** (*layers of isolation*): los cambios hechos en una capa no impactan a componentes de otras siempre que los contratos entre capas se mantengan, y cada capa es independiente de las demás y conoce poco o nada del interior de las otras (Richards, 2015, pp. 3–4).

Abrir una capa invierte esa garantía. Si la presentación puede acceder directamente a la persistencia, un cambio en la persistencia impacta a la vez a la capa de negocio y a la de presentación, produciendo "a very tightly coupled application with layer interdependencies" y una arquitectura "very brittle, as well as difficult and expensive to change" (Richards y Ford, 2020, cap. 10); el mismo argumento, con idéntica conclusión de que el sistema se vuelve "very hard and expensive to change", está en Richards (2015, p. 4). El *fast-lane reader* es exactamente ese mecanismo: declarar abiertas las capas de negocio y persistencia para que la presentación lea directamente la base de datos en consultas simples de recuperación (Richards y Ford, 2020, cap. 10).

A largo plazo, la mantenibilidad se juega en la gobernanza de esa decisión: "Failure to document or properly communicate which layers in the architecture are open and closed (and why) usually results in tightly coupled and brittle architectures that are very difficult to test, maintain, and deploy" (Richards, 2015, p. 5). Con capas abiertas, el aislamiento deja de ser una garantía de la arquitectura y pasa a depender de la disciplina del equipo: cada atajo autorizado multiplica los caminos posibles de cada solicitud, de modo que conocer el impacto de un cambio exige rastrear todos los flujos que cruzan capas. No sorprende que las mismas

fuentes adviertan que, a medida que crecen las aplicaciones en capas, “characteristics like maintainability, agility, testability, and deployability are adversely affected” (Richards y Ford, 2020, cap. 10). La cuenta, en síntesis: la capa abierta compra rendimiento local con la moneda de la mantenibilidad, justo el atributo que se quiere conservar a largo plazo.

6.2. Capas abiertas, sinkhole anti-pattern y la regla del 80/20

El **architecture sinkhole anti-pattern** ocurre cuando las solicitudes atraviesan las capas como *pass-through* simple, sin lógica de negocio en ninguna de ellas (Richards, 2015, p. 7); (Richards y Ford, 2020, cap. 10). El ejemplo canónico: la presentación pide datos básicos de un cliente; negocio y persistencia no hacen más que reenviar la petición hasta una consulta SQL trivial; los datos suben por la pila sin agregarse, calcularse ni transformarse. El coste no es inocuo: “unnecessary object instantiation and processing, impacting both memory consumption and performance” (Richards y Ford, 2020, cap. 10).

Conviene precisar la dirección causal, porque las fuentes no dicen que las capas abiertas *causen* el sinkhole: dicen que son la **reacción** propuesta cuando el sinkhole domina el tráfico. Richards (2015, p. 7) sugiere considerar capas abiertas cuando la proporción de peticiones *pass-through* se invierte, advirtiendo de inmediato que “it will be more difficult to control change due to the lack of layer isolation”; Richards y Ford (2020, cap. 10) presentan abrir todas las capas como “another approach to solving the architecture sinkhole anti-pattern”, con el mismo *trade-off* de “increased difficulty in managing change within the architecture”.

¿Por qué esa «solución» conduce de regreso al antipatrón? Porque no elimina su causa —capas intermedias vacías de lógica— sino que **institucionaliza el rodeo**: la capa hueca sigue existiendo, ahora como *pass-through* permanente para las peticiones que aún la atraviesan, y el atajo sancionado elimina el incentivo para escribir en ella lógica de negocio. Cuantas más peticiones la salten, más se vacía; el sinkhole deja de ser un escenario puntual y se vuelve el estado crónico de la arquitectura. El resultado combina lo peor de ambos mundos: el coste de rendimiento del sinkhole y el acoplamiento de las capas abiertas.

La **regla del 80/20** desarma la propuesta del estudiante porque no es una perilla de optimización sino un **diagnóstico de encaje del estilo**. Es típico que en torno al 20 % de las solicitudes sean *pass-through* y el 80 % porten lógica de negocio; el problema aparece cuando la proporción se invierte: “if 80 percent of the requests are sinkholes, it [is] a good indicator that the layered architecture is not the correct architecture style for the problem domain” (Richards y Ford, 2020, cap. 10); (Richards, 2015, p. 7). Es decir: si un sistema necesita capas abiertas y *fast-lanes* por doquier para ser viable, la regla no está autorizando la optimización; está dictaminando que la N-Capas no es la arquitectura correcta para ese dominio. La respuesta razonable al diagnóstico es cambiar de estilo, no optimizar el estilo equivocado, lo que conduce al tercer apartado.

6.3. Hexagonal y Limpia frente a N-Capas para alta escalabilidad y agilidad

Las calificaciones bajas de la N-Capas son estructurales, no accidentales. La agilidad general es baja porque hacer cambios sigue siendo "cumbersome and time-consuming [...] because of the monolithic nature of most implementations as well as the tight coupling of components" (Richards, 2015, p. 8). El despliegue es bajo porque "One small change to a component can require a redeployment of the entire application", incompatible con la entrega continua (Richards, 2015, p. 8); Richards y Ford (2020, cap. 10) puntúan la desplegabilidad como "very low" —un cambio de tres líneas obliga a redesplegar la unidad completa— y la elasticidad y escalabilidad como "very low (one star) [...] primarily due to monolithic deployments and the lack of architectural modularity". La escalabilidad es baja por "the trend toward tightly coupled and monolithic implementations" (Richards, 2015, p. 9).

Característica	**N-Capas**	**Hexagonal**	**Limpia**
Agilidad general	Baja: cambios lentos y engorrosos por la naturaleza monolítica y el acoplamiento (Richards, 2015, p. 8)	Alta: el núcleo se desarrolla y prueba aislado de dispositivos y bases de datos (Cockburn, 2005)	Alta: las reglas de negocio se prueban sin interfaz, base de datos ni servidor (Martin, 2017, p. 202)
Facilidad de despliegue	Muy baja: un cambio de tres líneas obliga a redesplegar la unidad completa (Richards y Ford, 2020, cap. 10)	Alta: cada adaptador es sustituible sin tocar el núcleo (Cockburn, 2005)	Alta: los detalles son *plugins* reemplazables con mínimo esfuerzo (Martin, 2017, p. 209)
Escalabilidad	Muy baja: despliegue monolítico y falta de modularidad arquitectónica (Richards, 2015, p. 9); (Richards y Ford, 2020, cap. 10)	Alta: la partición por dominio permite escalar una capacidad sin arrastrar la pila (Palermo, 2008a)	Alta: el núcleo no depende de la infraestructura que se escala (Martin, 2017, p. 203)

Tabla 6.1: Atributos de calidad: N-Capas frente a Hexagonal y Limpia. Las valoraciones de la columna N-Capas son las puntuaciones publicadas en Richards (2015, pp. 8–9) y Richards y Ford (2020, cap. 10). Hexagonal y Limpia no están puntuadas en esas fuentes: sus celdas son valoración propia, derivada de la Regla de Dependencia y de las propiedades que las fuentes citadas en cada celda atribuyen a esas arquitecturas.

Las optimizaciones del estudiante no tocan ninguna de esas tres causas. Actúan sobre el **camino de las solicitudes**, no sobre la **dirección de las dependencias** —el negocio sigue dependiendo de la persistencia—, ni sobre la **unidad monolítica de despliegue** —un cambio sigue redesplegando todo—, ni sobre el **tipo de partición**: la N-Capas es una arquitectura técnicamente particionada, en la que el dominio (por ejemplo, `customer`) está disperso por todas las capas, de modo que "a domain-driven design approach does not work as well with the layered architecture style" (Richards y Ford, 2020, cap. 10). Peor: abrir capas añade dependencias transversales, es decir, incrementa exactamente el acoplamiento que motiva las calificaciones bajas. El propio libro que documenta estas técnicas lo dice del

rendimiento: puede paliarse "through caching, multithreading, and the like, but it is not a natural characteristic of this architecture style" (Richards y Ford, 2020, cap. 10). Las capas abiertas son lo mismo: un parche que no es natural del estilo.

Las arquitecturas de dominio atacan la raíz, porque **invierten la dirección de las dependencias**. La Regla de Dependencia de la Arquitectura Limpia es explícita: "Source code dependencies must point only inward, toward higher-level policies. Nothing in an inner circle can know anything at all about something in an outer circle" (Martin, 2017, p. 203), una regla que se aplica siempre, con independencia del sentido del flujo de control, y que cruza cada límite mediante el DIP (Martin, 2017, p. 206). En la Hexagonal de Cockburn, el núcleo define *puertos* y los *adaptadores* externos los implementan, con el objetivo explícito de que la aplicación se desarrolle y pruebe "in isolation from its eventual run-time devices and databases", permaneciendo "blissfully ignorant of the nature of the input device" (Cockburn, 2005). En la Onion de Palermo, el Modelo de Dominio ocupa el centro, "only coupled to itself", con las interfaces —por ejemplo, las de repositorio— en el núcleo y el comportamiento que toca bases de datos fuera (Palermo, 2008a).

De esa única decisión estructural se derivan los atributos exigidos:

- **Agilidad.** Las reglas de negocio se prueban "without the UI, database, web server, or any other external element" (Martin, 2017, p. 202): ciclos de prueba cortos y deterministas. Y los cambios de detalle no se propagan: al quedar obsoleto un elemento externo —base de datos, framework web—, "you can replace those obsolete elements with a minimum of fuss" (Martin, 2017, p. 209). En N-Capas, en cambio, un cambio menor en persistencia golpea a negocio y a presentación (Richards, 2015, p. 4).

- **Desplegabilidad.** Los detalles son *plugins* del núcleo: sustituir o desplegar un adaptador no obliga a tocar el dominio; en N-Capas, el cambio de tres líneas redespliega la unidad entera (Richards y Ford, 2020, cap. 10); (Richards, 2015, p. 8).

- **Escalabilidad.** Al particionarse por dominio y no por capas técnicas, cada capacidad de negocio puede evolucionar, probarse, desplegarse y escalarse sin arrastrar la pila completa; en N-Capas, escalar una funcionalidad caliente equivale a escalar el monolito entero (Richards, 2015, p. 9).

Y la palabra «inherentemente» del enunciado es exacta: la diferencia entre ambos estilos no es de grado sino de **dirección de las dependencias**, una propiedad definitoria y no un parámetro ajustable. Abrir una capa no invierte dependencia alguna: solo autoriza más aristas en la misma dirección hacia afuera. Para que una N-Capas cumpliera la Regla de Dependencia habría que mover las interfaces al núcleo y hacer que la infraestructura las implemente, es decir, reconstruirla como Hexagonal o Limpia. Por eso, ante requisitos de alta escalabilidad y agilidad, las arquitecturas de dominio no son «una optimización más» de la N-Capas: son la corrección de la causa que ninguna optimización de la N-Capas puede alcanzar.

Conclusiones

El análisis de las preguntas lleva a una misma idea: lo que distingue a una buena arquitectura no es el número de capas ni la forma del dibujo, sino que la lógica de negocio no

dependa de los detalles técnicos. Cuando se consigue, el sistema se puede probar y cambiar por partes sin que cada modificación se propague a todo el código. Esa independencia, sin embargo, tiene un coste de construcción y mantenimiento (Martin, 2017, p. 218), por lo que la arquitectura adecuada depende del tamaño del sistema y de cuánto se espera que cambie.

Referencias bibliográficas

- Cockburn, Alistair (4 de sep. de 2005). *Hexagonal Architecture (Ports and Adapters)*. URL: https://alistair.cockburn.us/hexagonal-architecture/ (visitado 15-07-2025).
- De la Torre Llorente, César et al. (2010). *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0 (Borrador)*. Microsoft Ibérica. ISBN: 978-84-936696-3-8.
- Martin, Robert C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. ISBN: 978-0-13-449416-6.
- Palermo, Jeffrey (29 de jul. de 2008a). *The Onion Architecture: part 1*. URL: https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/ (visitado 15-07-2025).
- — (30 de jul. de 2008b). *The Onion Architecture: part 2*. URL: https://jeffreypalermo.com/2008/07/the-onion-architecture-part-2/ (visitado 15-07-2025).
- — (4 de ago. de 2008c). *The Onion Architecture: part 3*. URL: https://jeffreypalermo.com/2008/08/the-onion-architecture-part-3/ (visitado 15-07-2025).
- Richards, Mark (2015). *Software Architecture Patterns*. O'Reilly Media.
- Richards, Mark y Neal Ford (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media. ISBN: 978-1-492-04345-4.

Uso de IA

Para la elaboración de este seminario, el equipo utilizó dos herramientas de inteligencia artificial. Heily Rodríguez Rodríguez y Ernesto Alejandro Soler Choong usaron **Claude**, de Anthropic, con el modelo **Claude Opus 5.5**. Enrique González Moreira usó **OpenCode** con el modelo **GLM 5.3**.

Las herramientas se emplearon para buscar en la bibliografía recomendada los capítulos, epígrafes y páginas útiles para cada pregunta, y para comprobar que cada cita remite a la página o sección correcta. También se usaron para redactar borradores de las respuestas, mejorar la formalidad y la claridad de la redacción, y traducir al español ideas y fragmentos de las fuentes, que en su mayoría están en inglés. Por último, sirvieron para ayudar a cumplir los requisitos formales de la orientación, como la portada, el estilo de citas y la correspondencia entre las citas del texto y las referencias, y para generar el código LaTeX del documento, incluidos los diagramas.

Los borradores generados se utilizaron como base de las respuestas, que el equipo revisó, corrigió y reescribió hasta su versión final. Las citas se compararon con los textos originales

de la bibliografía. Las traducciones sirvieron para comprender y parafrasear las ideas de los autores, y los fragmentos traducidos que aparecen entre comillas se contrastaron con el texto original. El código LaTeX y los diagramas se revisaron y ajustaron hasta obtener el resultado deseado.

Todo el contenido generado fue revisado, verificado y adaptado por el equipo, quien asume la responsabilidad final del documento.